

IF440: Artificial Intelligence

Topic 12: Genetic Algorithm

Moeljono Widjaja, Ph.D.
Yaya Suryana, Ph.D.

Universitas Multimedia Nusantara
Jl. Scientia Boulevard, Gading Serpong
Tangerang, Banten-15811 Indonesia

Odd Semester 2025-2026

Course Sub Learning Outcomes (Sub-CLO):



SubCLO-0221: Students are able to apply knowledge-based reasoning, probabilistic reasoning and soft computing (C3).



Review

1 Introduction to Fuzzy Logic

- Crisp Sets
- Fuzzy Set Theory
- Operations on Fuzzy Sets
- Membership Functions

2 Fuzzy Inference System

- Fuzzification
- Defuzzification
- Fuzzy Rules

3 Exercise



Pre-class Assignment

- Kie Codes: Genetic Algorithms Explained By Examplec
https://youtu.be/uQj5UNhCPuo?si=3m7wR5H2oF5L_Vio
- Siraj Raval: Genetic Algorithm in Artificial Intelligence - The Math of Intelligence
<https://www.youtube.com/watch?v=rGWBo0JGf50>



Outline

- 1 Introduction
- 2 Basic Components
- 3 Algorithm Flow
- 4 Detailed Examples
- 5 Practical Example
- 6 Strengths and Limitations
- 7 Applications

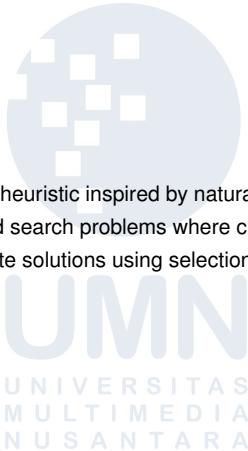




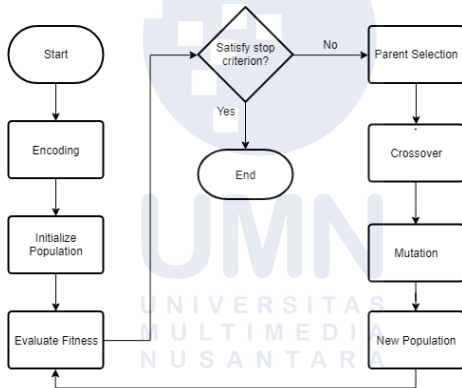
Introduction

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

What is a Genetic Algorithm (GA)?

- 
- A population-based metaheuristic inspired by natural selection.
 - Used for optimization and search problems where classical methods struggle.
 - Key idea: evolve candidate solutions using selection, crossover, and mutation.

GENETIC ALGORITHM



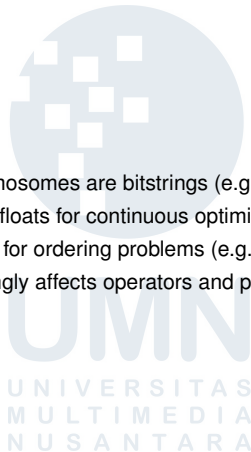


Basic Components

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Representation (Encoding)

- **Binary encoding:** chromosomes are bitstrings (e.g. 010110).
- **Real-valued encoding:** floats for continuous optimization.
- **Permutation encoding:** for ordering problems (e.g. TSP).
- Choice of encoding strongly affects operators and performance.



ENCODING

- 1. Binary Encoding** – every chromosome is a **string of bits** (0 or 1).

Chromosome A	101101100011
Chromosome B	010011001100

- 2. Permutation Encoding** – every chromosome is a **string of numbers**, which represent the number in the sequence (ordering problems).

Chromosome A	1 5 3 2 6 4 7 9 8
Chromosome B	8 5 6 7 2 3 1 4 9

ENCODING

3. **Value Encoding** – every chromosome is a **string of some values**. Values can be form of numbers, real numbers, or characters.

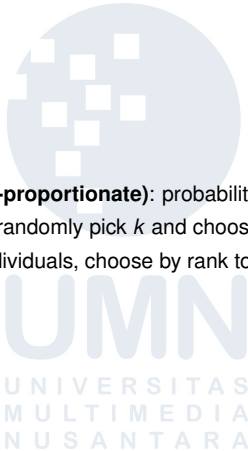
Chromosome A	1.2321 5.4367 2.4243 0.4563 3.5432
Chromosome B	(back) (back) (right) (forward) (left)

Fitness Function

- **Fitness function** - evaluation function that we are optimizing (objective function + penalty function).
- Choosing the right fitness function: an important genetic algorithm design issue
- Maps a candidate solution to a scalar fitness value.
- Determines reproductive success (higher fitness = higher chance of selection).
- Must align with the optimization objective; may require scaling or normalization.

UNIVERSITAS
MULTIMEDIA
NUSANTARA

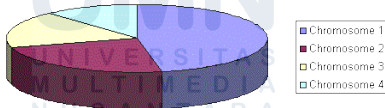
Selection Operators

- 
- **Roulette-wheel (fitness-proportionate)**: probability proportional to fitness.
 - **Tournament selection**: randomly pick k and choose best — widely used.
 - **Rank selection**: rank individuals, choose by rank to avoid premature domination.

PARENT SELECTION

1. Roulette Wheel Selection

- Selection in this method is **proportionate** to the fitness of chromosome.
- Higher the fitness of chromosome, higher the chances of getting selected.
- Generate random numbers until **desired number of chromosomes selected**.



PARENT SELECTION

2. Rank Selection

- After sorting the chromosomes on the basis of their fitness, **rank is assigned to them.**
- The best chromosome gets rank N (number of chromosomes) and the worst chromosome gets rank 1.

Chromosome	Fitness Value	Rank
1	8.2	4
2	3.2	2
3	1.4	1
4	4.2	3

PARENT SELECTION

3. Tournament Selection

- 2 chromosomes of current population chosen randomly.
- Whichever has the **higher fitness** is copied into a mating pool.
- Repeated until the mating pool is full.



PARENT SELECTION

3. Tournament Selection

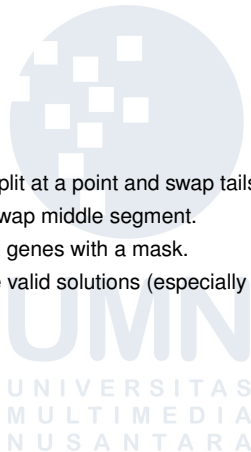
Chromosome	Fitness Value
1	8.2
2	3.2
3	1.4
4	4.2
5	1.2
6	2.3
7	5.8

Pick the higher fitness as parent

7

Crossover (Recombination)

- **One-point crossover:** split at a point and swap tails.
- **Two-point crossover:** swap middle segment.
- **Uniform crossover:** mix genes with a mask.
- Operators must preserve valid solutions (especially for permutations).



CROSSOVER

P_c = crossover rate/crossover probability

1. Single Point Crossover

- A cross-site is selected randomly along the length of the mated strings.
- Bits next to the cross-site are exchanged.

Parent 1

1	0	1	1	1	1	1	1
---	---	---	---	---	---	---	---

Parent 2

0	1	0	1	0	0	0	1
---	---	---	---	---	---	---	---

Offspring 1

1	0	1	1	1	0	0	1
---	---	---	---	---	---	---	---

Offspring 2

0	1	0	1	0	1	1	1
---	---	---	---	---	---	---	---

CROSSOVER

2. Two Point Crossover

- A cross-site is selected randomly along the length of the mated strings.
- Bits next to the cross-site are exchanged.

Parent 1

1	0	1	1	1	1	1	1
---	---	---	---	---	---	---	---

Parent 2

0	1	0	1	0	0	0	1
---	---	---	---	---	---	---	---

Offspring 1

1	0	1	1	0	0	1	1
---	---	---	---	---	---	---	---

Offspring 2

0	1	0	1	1	1	0	1
---	---	---	---	---	---	---	---

CROSSOVER

3. Uniform Crossover

- Bits are randomly copied from the first or from the second parent.

Mask: 01100110 (randomly generated)

If 1 is from Parent 1 else Parent 2

Parent 1

1	0	1	0	0	0	1	1
---	---	---	---	---	---	---	---

Parent 2

0	0	1	1	0	1	0	0
---	---	---	---	---	---	---	---

Offspring 1

0	0	1	1	0	0	1	0
---	---	---	---	---	---	---	---

Offspring 2

1	0	1	0	0	1	0	1
---	---	---	---	---	---	---	---

CROSSOVER

4. Arithmetic Crossover

- Some arithmetic operation is performed to make a new offspring

$$C_{i^{gen+1}} = a.C_{i^{gen}} + (1-a) C_{j^{gen}}$$

$$C_{j^{gen+1}} = a.C_{j^{gen}} + (1-a) C_{i^{gen}}$$

$$a = 0.2$$

Parent 1

0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8
-----	-----	-----	-----	-----	-----	-----	-----

Offspring 1

0.26	0.2	0.3	0.24	0.34	0.28	0.38	0.32
------	-----	-----	------	------	------	------	------

Parent 2

0.3	0.2	0.3	0.2	0.3	0.2	0.3	0.2
-----	-----	-----	-----	-----	-----	-----	-----

Offspring 2

0.14	0.2	0.3	0.36	0.46	0.52	0.62	0.68
------	-----	-----	------	------	------	------	------

Mutation

- Introduces random changes to chromosomes to maintain diversity.
- Examples: bit-flip (binary), gaussian perturbation (real-valued), swap/insert (permutation).
- Mutation rate controls exploration vs. exploitation.



MUTATION

P_m = mutation rate/mutation probability

1. Bit Inversion

- Selected bits are inverted

Offspring 1

0	0	1	1	0	0	1	0
---	---	---	---	---	---	---	---

Offspring 2

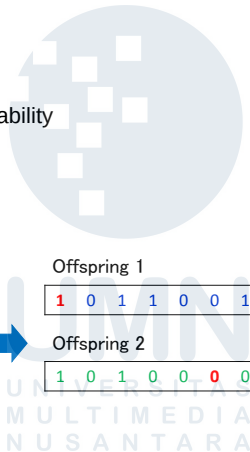
1	0	1	0	0	1	0	1
---	---	---	---	---	---	---	---

Offspring 1

1	0	1	1	0	0	1	0
---	---	---	---	---	---	---	---

Offspring 2

1	0	1	0	0	0	0	1
---	---	---	---	---	---	---	---



MUTATION

2. Order Changing

- Two numbers are selected and exchanged

Offspring 1

0	0	1	1	0	0	1	0
---	---	---	---	---	---	---	---

Offspring 2

1	0	1	0	0	1	0	1
---	---	---	---	---	---	---	---



Offspring 1

0	1	1	1	0	0	0	0
---	---	---	---	---	---	---	---

Offspring 2

1	0	1	0	0	1	0	1
---	---	---	---	---	---	---	---

UNIVERSITAS
MULTIMEDIA
NUSANTARA

Replacement / Survival Strategies

- **Generational replacement:** replace whole population each generation.
- **Steady-state replacement:** replace a few individuals at a time.
- **Elitism:** carry best individuals to next generation to preserve best-so-far.



Algorithm Flow



Standard GA Pseudocode

```
1 Initialize population P
2 Evaluate fitness of P
3 while not termination_condition:
4     parents = selection(P)
5     offspring = crossover(parents)
6     offspring = mutate(offspring)
7     evaluate(offspring)
8     P = replacement(P, offspring)
9 return best_solution(P)
```

Algoritmo: GA pseudocode

UNIVERSITAS
MULTIMEDIA
NUSANTARA

Parameter Choices

- Population size (e.g. 20–200)
- Evaluation function
- Crossover probability (commonly 0.6–0.9)
- Mutation probability (low, e.g. 0.001–0.1 depending on encoding)
- Selection pressure (tournament size, scaling)
- Termination criteria: max generations, fitness threshold, stagnation.

UNIVERSITAS
MULTIMEDIA
NUSANTARA



Detailed Examples

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

INITIALIZE POPULATION (case 1)

Generation 0

Chromosome Index	Chromosome String
1	00000110
2	11101110
3	00100000
4	00110010

$N = 4$

$l = 8$

UNIVERSITAS
MULTIMEDIA
NUSANTARA

EVALUATE FITNESS (case 1)

Generation 0

Chromosome Index	Chromosome String	Fitness = number of ones in the string
1	00000110	$f_1 = 2$
2	11101110	$f_2 = 6$
3	00100000	$f_3 = 1$
4	00110010	$f_4 = 3$

$N = 4$

$l = 8$

UNIVERSITAS
MULTIMEDIA
NUSANTARA

PARENT SELECTION (case 1)

Generation 0 : Roulette Wheel

Chromosome Index	Chromosome String	Fitness	Selection Probability	Cumulative Probability	Probability Distribution Function
1	00000110	$f_1 = 2$	$\frac{2}{12}=0,17$	0,17	0 – 0,17
2	11101110	$f_2 = 6$	$\frac{6}{12}=0,5$	0,67	0,17 – 0,67
3	00100000	$f_3 = 1$	$\frac{1}{12}=0,08$	0,75	0,67 – 0,75
4	00110010	$f_4 = 3$	$\frac{3}{12}=0,25$	1	0,75 – 1

Total fitness = 12

4 random numbers: **0,57; 0,10; 0,43; 0,8**

1st pair: strings 2 and 1

2nd pair: strings 2 and 4

CROSSOVER (case 1)

Generation 0

$$P_c = 0,7$$

Chromosome Index	Chromosome String	Fitness	Selection Probability	Cumulative Probability	Probability Distribution Function
1	00000110	$f_1 = 2$	$\frac{2}{12} = 0,17$	0,17	0 – 0,17
2	11101110	$f_2 = 6$	$\frac{6}{12} = 0,5$	0,67	0,17 – 0,67
3	00100000	$f_3 = 1$	$\frac{1}{12} = 0,08$	0,75	0,67 – 0,75
4	00110010	$f_4 = 3$	$\frac{3}{12} = 0,25$	1	0,75 – 1

1st pair: strings 2 and 1 $\rightarrow r = 0,4$

1 1 1 0 1 1 1 0
0 0 0 0 0 1 1 0

Apply crossover

2nd pair: strings 2 and 4 $\rightarrow r = 0,8$

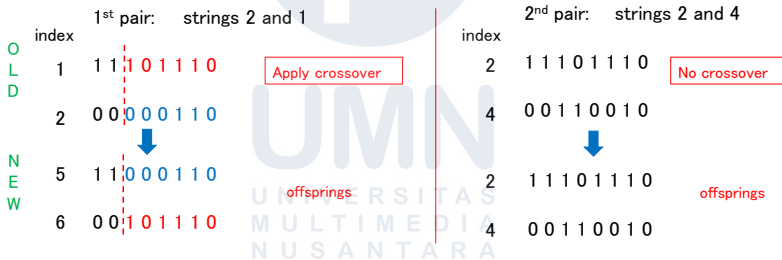
1 1 1 0 1 1 1 0
0 0 1 1 0 0 1 0

No crossover

CROSSOVER (case 1)

Generation 0

Random number 1– L to define crossover point, i.e. 2



MUTATION (case 1)

Generation 0

$$P_m = 0,001$$

1 st pair:			2 nd pair:		
index			index		
5	1 1 0 0 0 1 1 0	$r = 0,004 > P_m \rightarrow$ No Mutation	2	1 1 1 0 1 1 1 0	$r = 0,008 > P_m \rightarrow$ No Mutation
6	0 0 1 0 1 1 1 0	$r = 0,005 > P_m \rightarrow$ No Mutation	4	0 0 1 1 0 0 1 0	$r = 0,0005 < P_m \rightarrow$ Mutation
			7	0 0 1 1 0 1 1 0	

UNIVERSITAS
MULTIMEDIA
NUSANTARA

MUTATION (case 1)

Generation 0

1 st pair:			2 nd pair:		
OLD	index		index		
5	1 1 0 0 0 1 1 0	$r = 0,004 > P_m \rightarrow$ No Mutation	2	1 1 1 0 1 1 1 0	$r = 0,008 > P_m \rightarrow$ No Mutation
6	0 0 1 0 1 1 1 0	$r = 0,005 > P_m \rightarrow$ No Mutation	4	0 0 1 1 0 0 1 0	$r = 0,0005 < P_m \rightarrow$ Mutation
	↓		↓		
NEW	5	1 1 0 0 0 1 1 0	2	1 1 1 0 1 1 1 0	
	6	0 0 1 0 1 1 1 0	7	0 0 1 1 0 1 1 0	

NEW POPULATION (case 1)

Generation 0

Chromosome Index	Chromosome String	Fitness
1	00000110	$f_1 = 2$
2	11101110	$f_2 = 6$
3	00100000	$f_3 = 1$
4	00110010	$f_4 = 3$

Generation 1

Chromosome Index	Chromosome String
2	11101110
5	11000110
6	00101110
7	00110110

Discrete Example: Traveling Salesman Problem (TSP)

- **Objective:** Find the shortest route visiting each city once.
- **Encoding:** Permutation of cities (e.g. [A, C, D, B, E]).
- **Fitness:** $f = 1 / \text{total_distance}$.
- **Crossover:** Order Crossover (OX) or Partially Mapped Crossover (PMX).
- **Mutation:** Swap two cities or perform inversion.

UNIVERSITAS
MULTIMEDIA
NUSANTARA

Continuous Example: Function Optimization

- **Objective:** Maximize $f(x, y) = \sin(x) \cos(y) + 0.5(x + y)$.
- **Encoding:** Real-valued vector (x, y) .
- **Fitness:** Directly computed as $f(x, y)$.
- **Crossover:** Arithmetic crossover: $c_1 = \alpha p_1 + (1 - \alpha)p_2$.
- **Mutation:** Add Gaussian noise: $x' = x + \mathcal{N}(0, \sigma^2)$.

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA



Practical Example

UMN
UNIVERSITAS
MULTIMEDIA
NUSANTARA

Example: Maximize a function

- Problem: maximize $f(x) = x \sin(10\pi x) + 1.0$ for $x \in [0, 1]$.
- Use binary encoding (e.g. 16 bits) or real-coded GA.
- Show typical results: GA finds near-optimal peaks despite many local optima.



Python-like Implementation Sketch

```
1 # population: list of floats or bitstrings
2 for gen in range(max_gen):
3     fitness = [evaluate(ind) for ind in population]
4     new_pop = []
5     while len(new_pop) < pop_size:
6         p1 = tournament_select(population, fitness)
7         p2 = tournament_select(population, fitness)
8         c1, c2 = crossover(p1, p2)
9         c1 = mutate(c1); c2 = mutate(c2)
10        new_pop += [c1, c2]
11    population = replace(population, new_pop)
```

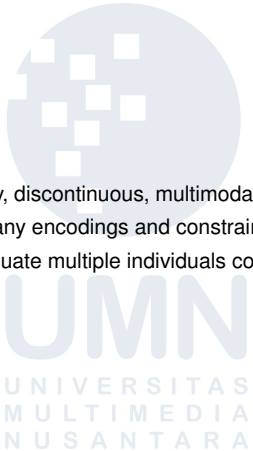
Algoritmo: Simple GA outline

UNIVERSITAS
MULTIMEDIA
NUSANTARA

Strengths and Limitations

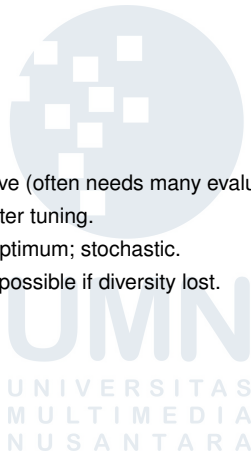


Strengths

- 
- Robust — works on noisy, discontinuous, multimodal functions.
 - Flexible — works with many encodings and constraints.
 - Easily parallelizable (evaluate multiple individuals concurrently).

Limitations

- Computationally expensive (often needs many evaluations).
- Requires careful parameter tuning.
- No guarantee of global optimum; stochastic.
- Premature convergence possible if diversity lost.

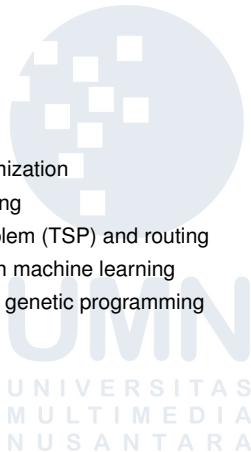


Applications



Common Applications

- Engineering design optimization
- Scheduling and timetabling
- Traveling Salesman Problem (TSP) and routing
- Hyperparameter tuning in machine learning
- Symbolic regression and genetic programming



Further Reading



J. H. Holland, **Adaptation in Natural and Artificial Systems**, 1975.



D. E. Goldberg, **Genetic Algorithms in Search, Optimization, and Machine Learning**, 1989.



A. E. Eiben, J. E. Smith, **Introduction to Evolutionary Computing**, 2nd ed., 2015.



Next Week

- Introduction to Reinforcement Learning
- Markov Decision Process
- Q Learning
- Summary



Vision and Mission

Vision

Becoming a leading Informatics Undergraduate Study Program that produces graduates with international insight who are competent in the field of computer science, have an entrepreneurial spirit, and have noble character.



Mission

- 1 Organising learning with the best technology and curriculum and supported by professional teaching staff.
- 2 Carry out research activities in the field of informatics to advance information science and technology.
- 3 Carry out community service activities based on information technology and science in the context of practising informatics science and technology society.

